GPIO Driver Implementation for LED and Push Button Control on STM32 Microcontroller

Melanie Picen – 35032, Sebastian Lizalde-Student 38241, Michelle Urbina-37964
CETYS Universidad

***Abstract*—This file implements the LED module for the STM32 microcontroller. It provides four functions to control a GPIO-connected LED: initialization, turn on, turn off, and toggle. The module relies on the GPIO driver layer to perform low-level register operations on PORT A, PIN 5.**

Keywords—List of document keywords (e.g., GPIO, MCU, etc.)

I. Introduction

This report presents the design of a GPIO-Based LED and Push Button Control system that incorporates the STM32F411RE microcontroller, referred to here as the STM32.

The objective of this Lab Assignment is to implement a layered driver architecture for the STM32 microcontroller, separating the system into three distinct layers: the Application Layer, the Driver Layer, and the Hardware Layer. This architecture allows each module to operate independently, improving code reusability and maintainability. The system implements two peripheral modules: a Push Button module that reads user input from PIN13 on Port C, and an LED module that controls an output LED on PIN5 of Port A. Together, these modules demonstrate a complete input-output interaction — when the push button is pressed, the LED turns on, and when released, the LED turns off.

The materials used for this Lab Assignment include the STM32F411RE Nucleo development board, the STM32CubeIDE development environment, and the CMSIS hardware abstraction libraries provided by ARM for low-level register access.

II. Process Description

Procedure of the Lab Assignment The development of this lab assignment was executed through a structured, modular, and collaborative approach. The procedure was divided into the following key stages to ensure proper hardware abstraction and clear module boundaries:

Build System and Environment Setup: The project began by establishing the compilation infrastructure from scratch to automate the build process. Instead of manually typing complex compilation commands every time a change was made, the team created a build system divided into two main files:

- **sources.mk (The Inventory):** This file acts as a centralized list of all the raw materials needed for the project, specifically tracking the source code files (e.g., main.c, led.c, push_button.c) and header directories.
- **Makefile (The Rulebook):** This file contains the automated instructions for the ARM GCC compiler (arm-none-eabi-gcc). It reads the files listed in sources.mk and translates them into machine-readable object files (.o), merging them into a final executable (app.elf). It also integrates a custom linker script (stm32f4.ld) and startup code (stm32.startup.c) to organize exactly where the program will live inside the microcontroller's FLASH and SRAM memory.

By structuring the project this way, the team could control the entire hardware deployment process using simple terminal commands:

- Running **make build** automatically compiles and links all modified code.
- Running **make clean** deletes old compilation files to prevent errors.
- Running **make flash** triggers OpenOCD to establish a physical connection with the development board, verify the memory, and upload the final executable directly into the microcontroller.

2. Core GPIO Driver Implementation (Task 1): The foundational layer of the project was the GPIO driver (GPIO_stm32f4.c/h). This module was developed to directly manipulate the MCU's memory-mapped registers, acting as the bridge between software commands and physical hardware.

- **Initialization and Clocking:** Functions like gpio_initPort() were implemented to enable the specific peripheral clock registers; the MCU requires this because ports are powered off by default to save energy.
- **Mode Configuration:** The gpio_setPinMode() function was created to write to the MODER register, defining whether a pin acts as an input, output, or alternate function.
- **State Control:** To manage outputs, gpio_setPin() and gpio_clearPin() were designed to interact with the BSRR register, allowing the system to change pin states atomically (without interrupting other processes). Meanwhile, gpio_readPin() accesses the IDR register to capture incoming digital signals.
- **Error Handling:** The implementation strictly followed the SRS (FR-1 through FR-10) and incorporated explicit error handling (e.g., returning DRIVER_STATUS_E_INVALID_PIN). The program needs this robust validation to prevent the MCU from attempting to access non-existent memory addresses or out-of-bounds pins, which would cause a hardware fault.

3. LED Module Development (Task 2): A dedicated hardware abstraction layer was created for the user LED (led.c/h). To guarantee strict modularity, this module was restricted from accessing hardware registers directly; it relied entirely on the core GPIO API.

- **led_init():** This function is required at system startup to configure the specific microcontroller pin connected to the LED as a digital output using the driver's mode functions.
- **led_on() and led_off():** These functions were implemented to explicitly drive the pin to a logic High or Low state. The program needs these to translate abstract application commands into the physical voltage changes required to illuminate or extinguish the LED.
- **led_toggle():** This function was designed to read the current state of the LED and invert it. It is essential for the application loop because it allows the system to blink the LED continuously without needing to store or track its previous state manually.

4. Push-Button Module Development (Task 3): Similarly, the Push-Button component (push_button.c/h) was encapsulated into its own module, utilizing the GPIO API to interface with the physical button.

- **button_init():** This function was implemented to configure the target pin strictly as a digital input, preparing the MCU to receive external voltage signals rather than outputting them.
- **button_get_state():** The module relies on this function to continuously sample the physical voltage on the pin. This function is critical for the program's logic because it translates the mechanical action of the user pressing the button into a digital boolean value (pressed or released) that the main application layer can evaluate to trigger events, such as toggling the LED.

Push-Button Debounce Finite State Machine (FSM)

The Problem of Mechanical Bouncing When a physical push-button is pressed, the internal metal contacts do not close perfectly at once. Instead, they bounce against each other for a few milliseconds, creating a rapid series of erratic HIGH and LOW signals. If the microcontroller reads this directly, a single human press might be interpreted as multiple consecutive presses. To solve this, a software-based Finite State Machine (FSM) was designed to act as a "debounce" filter.

System Inputs and Design Logic To accurately determine if a button press is real or just mechanical noise, the FSM evaluates two specific inputs:

1. **The Physical Signal (pinVal):** Represents the actual voltage on the pin (1 = button released, 0 = button pressed).
2. **The Time Threshold (count >= 50):** A digital counter acting as a timer. *Why were these two inputs chosen?* Relying solely on the physical signal is unreliable due to the bouncing effect. By combining the physical signal with a timer, the system can enforce a rule: the button must remain in the pressed state for a continuous, specific amount of time before the system accepts it as a valid human action.

Software Implementation Strategy (static variables) In the C code (push_button.c), the FSM state and the debounce counter are declared as static variables inside the button_get_state() function. This is a critical design decision. Standard local variables are destroyed and reset every time a function finishes executing. Because the main program continuously calls this function in an infinite loop, the function must "remember" what state it was in and what the count was during the previous cycle. The static keyword preserves this memory safely inside the function's scope without needing vulnerable global variables.

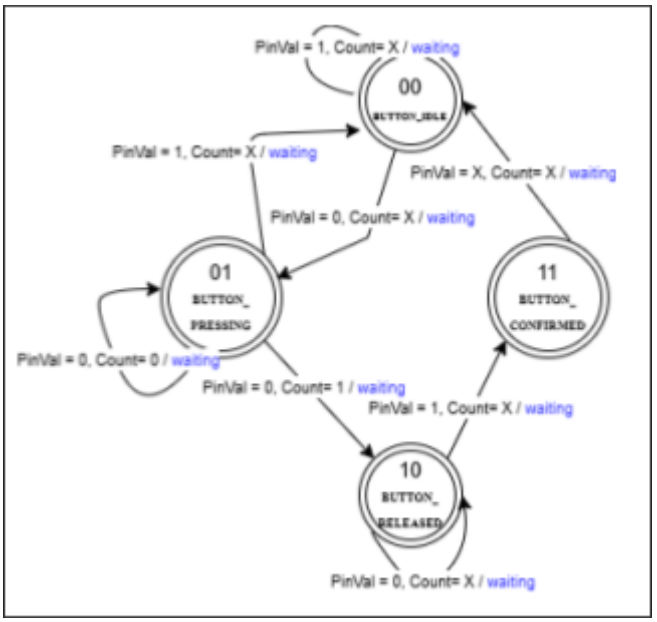

Figure 1. Finite State Machine diagram for the Push-Button debounce logic**.**

Each path has a label formatted as Input 1, Input 2 / Output. **The letter "X" (Don't Care):** It can be 0 or 1.

The Story of a Click: State Machine Execution

The system goes through four phases to ensure a "click" is real and not just a mechanical glitch:

- **State 00 (BUTTON_IDLE - On Standby):** The system is resting, just keeping watch. As long as no one touches the button (signal is 1), it stays here. If it detects a 0 (someone touched the button), it resets its timer and moves to the next step to investigate whether it was a real press or just interference.
- **State 01 (BUTTON_PRESSING - Verifying the Press):** This is where we filter out the "noise." The system waits here and counts.
    - If the button is released before the timer finishes, the system says: "False alarm,

that was just a metallic bounce," and goes back to standby.
    - If the button stays pressed and the timer reaches its goal (Count = 1), the press is mathematically confirmed as real, and we move forward.
- **State 10 (BUTTON_RELEASED - Waiting for the Release):** We already know a real press happened, but the system is smart: it doesn't want to register 50 clicks in a row just because you held your finger down. So, it waits here patiently. As soon as you let go of the button (signal changes back to 1), the system knows the physical movement is completely over and moves to the final state.
- **State 11 (BUTTON_CONFIRMED - Click Confirmed!):** The cycle is successfully completed. This is where the system tells the main program: "We have a perfect, clean click!". The system completely ignores the inputs, executes its output alert, and unconditionally resets itself back to the standby state (BUTTON_IDLE) so it is ready for the very next click.

5. Application Integration and Logic (Task 4): Finally, all individual modules were merged into main.c. The application layer was constructed by initializing one pin as an output (LED) and another as an input (Push-Button). Inside the main infinite loop, the system continuously polled the button state using button_get_state() and triggered led_toggle() upon a valid press. To ensure reliable operation and prevent false triggers from mechanical noise, software debounce logic was integrated into the button reading routine.

Challenges Faced Throughout the development process, the team encountered and resolved several technical and integration challenges:

- **Understanding Hardware Registers:** Before writing the GPIO logic, a significant challenge was comprehending the specific functions of the STM32 registers. It required careful analysis of the reference manual to understand how to correctly sequence the clock enabling, mode configuration (MODER), and atomic bit operations (BSRR) to configure the pins safely.
- **Enforcing Strict Modularity:** During the development of the LED and Push-Button modules, a key challenge was ensuring that these upper layers did not bypass the GPIO API. We had to strictly enforce that all state changes and initializations were routed through the functions defined in Task 1, rather than writing directly to registers in the application layer.
- **Integration and Naming Inconsistencies:** When merging the separate .c and .h files developed in parallel by different team members, we faced integration roadblocks. Minor inconsistencies in variable naming conventions, mismatched function signatures, and unaligned parameter definitions caused initial compilation errors. This required a collaborative code review session to standardize the interfaces and ensure seamless compatibility across all modules.

III. Results

The lab assignment was completed successfully. All four tasks defined in the assignment were implemented and validated on the STM32F411RE Nucleo-64 development board.

The GPIO driver was implemented following all functional requirements FR-1 through FR-10 specified in the SRS. The driver correctly enables peripheral clocks via gpio_initPort(), configures pin direction through the MODER register using gpio_setPinMode(), performs atomic pin state changes via the BSRR register with gpio_setPin() and gpio_clearPin(), and reads digital input signals through the IDR register with gpio_readPin(). Error handling was validated by testing invalid port and pin parameters, which returned DRIVER_STATUS_E_INVALID_PIN as expected.

The LED module was verified to correctly initialize PIN 5 on Port A as a digital output. The led_on(), led_off(), and led_toggle() functions operated as intended, driving the on-board LED without any direct register access at the application level, confirming successful hardware abstraction.

The push-button module, connected to PIN 13 on Port C, was validated through the software debounce FSM. Mechanical bouncing was effectively filtered: only sustained LOW signals lasting at least 50 polling cycles were accepted as valid presses, and the LED toggle event was triggered exclusively upon button release, preventing multiple triggers from a single press.

The integrated application in main.c demonstrated correct end-to-end behavior — each complete press-and-release cycle of the user push-button produced exactly one LED toggle event, with no false triggers observed during testing.

Before pushed	After pushed

IV. Conclusions

This lab assignment successfully achieved its primary objective: the design and implementation of a modular, hardware-abstracted GPIO driver for the STM32F411RE microcontroller. The layered architecture (separating the

Application Layer), Driver Layer, and Hardware Layer proved effective in isolating responsibilities across modules and improving both code reusability and maintainability.

The LED and push-button modules demonstrated that upper-level software can interact with hardware exclusively through a clean API, without requiring direct register manipulation at the application level. The software debounce mechanism, implemented as a four-state Finite State Machine with a counter-based time threshold, reliably filtered mechanical noise and ensured that each physical button press produced exactly one logical event.

From a technical standpoint, the assignment deepened the team's understanding of STM32 peripheral architecture, particularly the role of clock enabling, the MODER, BSRR, and IDR registers, and the importance of atomic operations in embedded systems. The use of static variables within the FSM function highlighted a practical strategy for preserving state across function calls without relying on global variables.

From a software engineering perspective, the challenges encountered during integration — such as naming inconsistencies and mismatched function signatures across parallel development branches — reinforced the importance of defining clear module interfaces before implementation begins. This experience illustrated how a well-structured SRS and SDD contribute directly to reducing integration friction in team-based embedded projects.

Link GitHub:

Sebastian:

Michelle:

Melanie: **https://github.com/melanie-hdez/MSE-Lab.git**

References

[1] D. V. Lindberg and H. K. H. Lee, "Optimization under constraints by applying an asymmetric entropy measure," *J. Comput. Graph. Statist.*, vol. 24, no. 2, pp. 379–393, Jun. 2015, doi: 10.1080/10618600.2014.901225.

[2] B. Rieder, *Engines of Order: A Mechanology of Algorithmic Techniques*. Amsterdam, Netherlands: Amsterdam Univ. Press, 2020.

[3] I. Boglaev, "A numerical method for solving nonlinear integro-differential equations of Fredholm type," *J. Comput. Math.*, vol. 34, no. 3, pp. 262–284, May 2016, doi: 10.4208/jcm.1512-m2015-0241.